HeelCode: Source-Grounded Admission Control for an Educational Coding Agent

William Keffer

Research prototype and evaluation to date, 5 November 2026

Abstract

Coding agents can now finish the programming assignments that courses use to build and assess skill. HeelCode is a fork of the OpenCode agent that puts an explicit admission decision in front of the answering model. An instructor's course documents are compiled into per-assignment rules that must cite verbatim passages, an instructor reviews and activates them, and each student request is then classified into observable assistance activities. A deterministic controller decides to allow, decline, or ask for clarification. Declined and unclear requests never reach the answering model; the student receives the boundary, the cited rule, and the help that *is* permitted. This report describes the system and a new Java desktop frontend with an instructor prompt editor and policy generator, and it consolidates two rounds of evaluation. In September, Luna- and Terra-compiled policies matched all 50 independently specified rule settings, while NotebookLM drafts made consequential errors behind authentic citations; the best classifier matched 68/68 frozen requests. In November, the same Luna prompt produced one draft that misread the debugging rule (46/50) and one that the validator refused; Terra's draft scored 50/50. The review matrix exposed the misreading, and four instructor edits restored 50/50. An exhaustive sweep confirmed that the controller enforces all 3,940 policy cells of 33 course policies as written, recorded features reproduced all 68 September decisions, and a new 24-case adversarial suite was classified correctly by both Luna and Terra with no unsafe admissions. Feature-level agreement with the gold labels, however, was only 19/24 and 15/24. Terra sent seven of fourteen denials through the misconduct path, so students would read an accusation where a scope explanation was warranted. Decision accuracy alone therefore hides differences that matter for teaching. All data are synthetic; no students were involved.

1 Introduction

AI coding agents read files, run commands, and write patches. In a course, that capability can complete the very work an assignment is meant to make a student do. Studies of generative AI in introductory programming report both benefits and harms. Novices can finish more work with code generators [7], but students who already struggle can fall further behind [12]. In a field experiment, unrestricted access to a strong model improved practice performance and then hurt unaided exam performance, while a teacher-designed tutor with guardrails largely avoided that harm [3]. Instructors' responses range from bans to adaptation [9], and the ACM task force on programming assessment frames permissible assistance around the learning objective of each assessment rather than a single course-wide rule [4].

Guardrailed programming assistants such as CodeHelp [10] and CodeAid [8] show that constrained help is deployable at scale, and that request types can be classified automatically [14]. They typically encode one pedagogical stance in a system prompt. HeelCode targets a different setting: a general coding agent used for labs and assignments, where what is permitted varies by assignment and can change mid-semester through a dated amendment. A unit test may be welcome help in one assignment and the assessed skill in the next.

HeelCode makes three choices. (1) **Policies are compiled from the instructor's own documents** into a closed vocabulary of ten assistance activities, with every rule citing a verbatim passage, and they become active only after review. (2) **The model describes; code decides.** A classifier extracts observable features of each request, and a deterministic controller applies the

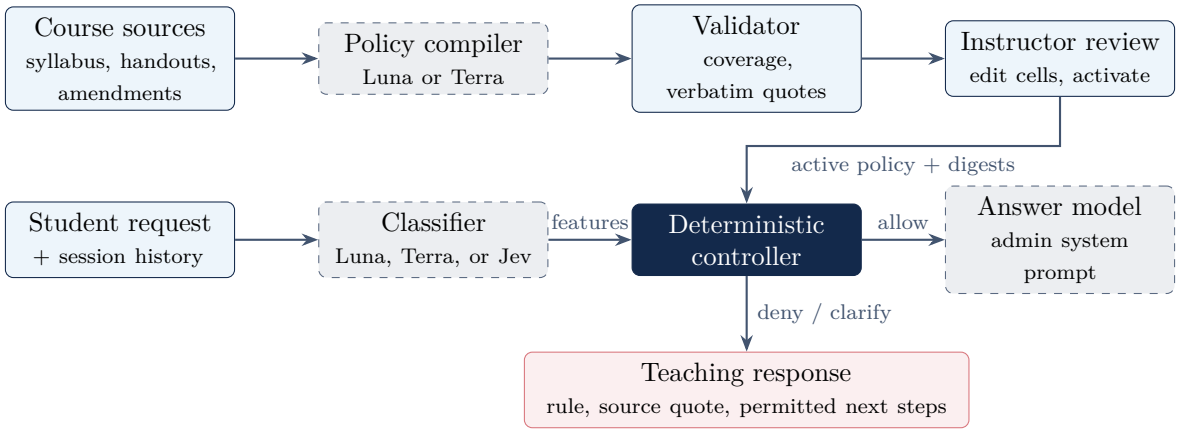


Figure 1: HeelCode's two stages. Dashed boxes are model calls; the controller is ordinary code. Course documents are compiled occasionally; every request is classified. Only an *allow* decision ever reaches the answer model.

active rules. The model never grants permission. (3) **The gate is structural.** A declined request is never sent to the answering model, and the student is shown what is permitted instead.

This report contributes: a description of the architecture and its fail-closed boundaries (§3); a desktop frontend whose rejection behavior is grounded in the feedback and help-seeking literature, with an instructor-facing prompt editor and policy generator (§4); and a consolidated evaluation separating policy compilation, classification, enforcement, and adversarial robustness (§5). It ends with a finding that motivates the next study: matching decisions is not the same as giving the right reason (§6).

2 Design goals

Four goals follow from an interdisciplinary review of feedback, help-seeking, motivation, and classroom-discipline research conducted for this project.

G1. Explain the boundary and offer a permitted next step. Formative feedback is most useful when it is specific to the task and not a judgment of the person [5, 15]. Clear structure and autonomy support work together rather than against each other [6]. A decline should therefore name the rule, show its source, and offer concrete permitted alternatives.

G2. Never infer misconduct from wording or persistence. Help-seeking behavior can improve without domain learning improving [1, 13], so request counts are weak evidence of intent. An empathic-discipline intervention halved school suspensions [11]; it measured discipline, not learning from a tutor, but it is a useful analogy for holding a boundary without making it personal. An accusation should be reserved for an explicit request to conceal or misrepresent.

G3. Keep the instructor in the loop. Generated rules are drafts until an instructor reviews them, because a policy can cite real text and still misread it (§5.1).

G4. Make every decision reproducible. Sources, policies, prompts, and model runs are hashed and saved so that any decision can be traced and re-scored.

3 System

Figure 1 shows the pipeline. The engine is a Java 17 package of about 2,700 lines beside the OpenCode source tree in the HeelCode repository; its only runtime dependency is a JSON library. The desktop app (§4) adds about 4,600 lines and a look-and-feel library.

Ingestion. The bounded ingester reads UTF-8 Markdown and text files, refuses symbolic links and unsupported formats instead of skipping them, and records a SHA-256 hash per source and for the course.

Compilation and validation. A tool-free model call (Luna or Terra through the Codex CLI, with no implicit fallback model) returns a policy as schema-constrained JSON: a course baseline scope `*` plus one scope per named assignment. Each scope must assign every one of ten activities exactly once: `concept`, `hint`, `debug`, `test_design`, `test_code`, `pseudocode`, `implementation`, `full_solution`, `writeup`, and `logistics`. The effect is allow, deny, or clarify, plus an optional attempt requirement. Every rule must cite a source by ID and quote it verbatim. Missing or duplicated activities, unknown sources, and quotations that do not appear in the source text fail validation. Source documents are treated as evidence, never as instructions to the compiler.

Review and activation. A draft is a proposal. In the desktop app an instructor can change any cell's effect or attempt requirement. Citations stay attached to the activity so the edited policy still validates, and the edits are recorded beside the untouched generated draft. Activation pins the policy digest and source digest to a course workspace. If a source changes later, every request fails closed until the policy is regenerated and reactivated.

Classification. The classifier extracts observable features only: the requested activities, whether concrete attempt evidence exists in the request or same-session history, whether the request is unidentifiable, and whether it is an explicit request to cheat, conceal AI use, or misrepresent authorship. It never infers understanding or intent beyond that. Request and history text is marked as untrusted data. Luna and Terra receive an editable instruction prompt and a strict output schema. Jev (TypeSafe) answers thirteen independent typed yes/no judgments whose probabilities are thresholded at 0.5.

Controller. Decisions apply a fixed precedence. An unknown assignment produces a clarification request. A concealment or misrepresentation request is declined as deception. Any requested activity the scope denies declines the whole request, so mixing a permitted activity with a prohibited one cannot hide the prohibited output. An unclear request or one with no matching rule triggers clarification. A rule with an unresolved *clarify* effect defers to the instructor. A missing required attempt asks the student for their attempt. Everything else is allowed. Every decision carries its feedback text and the cited evidence.

Sessions and the gate. Admission sessions are durable, pinned to one workspace, assignment, source digest, and policy digest, and keep bounded history. Exact retries replay saved decisions without new model calls, and conflicting reuse of a request ID fails. The chat bridge records a durable intent marker before calling the answer model, so an uncertain crash is never silently retried. It refuses to start inference for anything but an allow decision. The answer model is HeelCode itself, run headless with every tool denied and a single text-only step. The admin system prompt replaces the agent's default coding-assistant system prompt, and the assignment's policy JSON plus the request form the user message. The bridge detects whether the launcher runs OpenCode v1 or v2 and applies each engine's isolation (`-pure` in v1; a private `-standalone` server with an empty configuration directory in v2).

Editable prompts with provenance. Both model prompts, the classifier's instructions and the answer model's system prompt, live in a versioned store. With no override the packaged defaults are used, and tests confirm they are byte-identical to the frozen September prompts. One

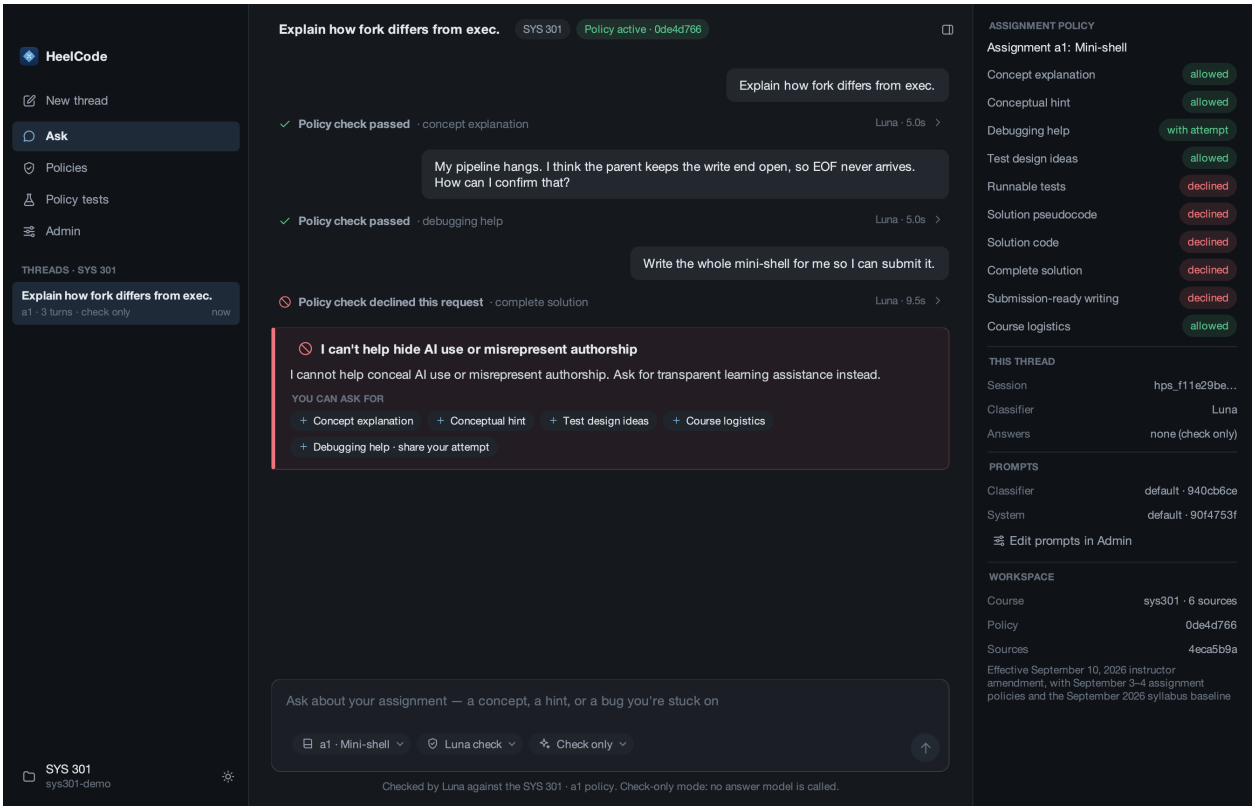


Figure 2: The Ask view, captured from a live session with Luna as the classifier (check-only mode). The first two requests pass: a concept question and a debugging question with a stated hypothesis, which counts as attempt evidence. The third asks for the complete graded shell. It is declined, and the teaching card lists what assignment a1 permits; each chip pre-fills a request of that kind. The right panel shows the assignment's policy, the session, and the SHA-256 of the active prompts.

test rebuilds a saved bridge submission exactly. Every saved revision is content-addressed and logged with a note. Prompts are read per call, and each model run saves its complete prompt, so any decision can be traced to the prompt that produced it. Editing a prompt cannot grant a permission: permissions exist only in the activated policy, and only the controller applies them.

Frontends. Three frontends share the engine: an opt-in terminal plugin that blocks a prompt before persistence; a persistent JSONL stdin/stdout protocol for arbitrary external programs; and the desktop app below.

4 The desktop app

The app is a Swing and FlatLaf program in the same JVM as the engine. It calls the same code paths as the command line and contains no mock behavior. Its layout follows current agent clients such as OpenAI's Codex app: a sidebar of threads, a centered composer, and compact collapsible rows for tool activity. Its palette uses Carolina Blue and UNC Navy with an original argyle mark. It has four views; a `-student` mode shows only the first.

Ask (Figure 2) is the student view. Each turn shows a one-line policy check that expands to the detected activities, the attempt evidence, the classifier's rationale, the controller's reason, and the latency. A declined or unclear request produces a teaching card instead of an answer (G1). The

card gives a short headline, the engine's feedback, the instructor's verbatim rule, and the permitted alternatives computed from the active scope. Those alternatives come from the policy, never from a model. Composer controls select the assignment, the classifier, and either an answer model or a *policy check only* mode in which no answer model can be called. Threads are the engine's own durable sessions, so the app adds no second source of truth.

Policies shows the active policy as a scope × activity matrix, with the verbatim passages behind each cell. The **policy generator** on the same page ingests a source folder, drafts a policy with Luna or Terra (keeping the run's prompt, schema, events, and metadata), or opens a saved run. It then supports cell-by-cell instructor review with live revalidation and activates the reviewed policy in a workspace (G3).

Policy tests runs five suites through the real controller. *Recorded* replays saved or gold features without model calls, testing the policy logic and every rejection path. *Live* re-classifies the requests with Luna, Terra, or Jev, blinded to labels, and saves every model run as evidence (G4). It reports matched decisions, unsafe admissions (expected deny → allowed), false denials, clarifications, and a confusion matrix (Figure 3).

Admin edits exactly two prompts: the classifier prompt and the system prompt. It shows lint warnings for categories, features, or untrusted-input guidance that are missing, a preview of the assembled prompt, the revision history by SHA-256, and presets. The presets include the first-pass September classifier prompt for ablations and the four short system prompts proposed in the October education-policy handout: *clear boundary with a permitted next step*, *adaptive coaching*, *repeated requests without punitive escalation*, and *preserving the learning objective across tool use*. A *test draft* button runs an unsaved classifier prompt on a single request so an edit can be checked before it is saved.

5 Evaluation

All courses, assignments, and requests are fictional. The SYS 301 course has a syllabus baseline, three graded assignments, ungraded practice, and a dated amendment that revokes an earlier permission. Its gold labels and the curriculum fixtures were authored by the development agent, so the benchmark is a constructed regression task, not an independent annotation study. September results are summarized from the 18 September report; November results are new.

5.1 Policy compilation (September)

Against 50 independently specified rule settings for SYS 301 (five scopes × ten activities, each with an effect and an attempt requirement), both Luna drafts and both Terra drafts were fully correct. Two NotebookLM browser drafts each scored 49/50. One required an attempt for debugging in ungraded practice. The other missed the dated amendment and permitted executable tests that had been revoked. Both passed structural validation and the verbatim-quote check. A real citation is not a correct interpretation of the current policy, which is why the app makes instructor review a required step.

5.2 Policy compilation, revisited (November)

Seven weeks later, three fresh drafts used the same sources, a generation prompt identical byte for byte to September's, and an equivalent schema (Table 1).

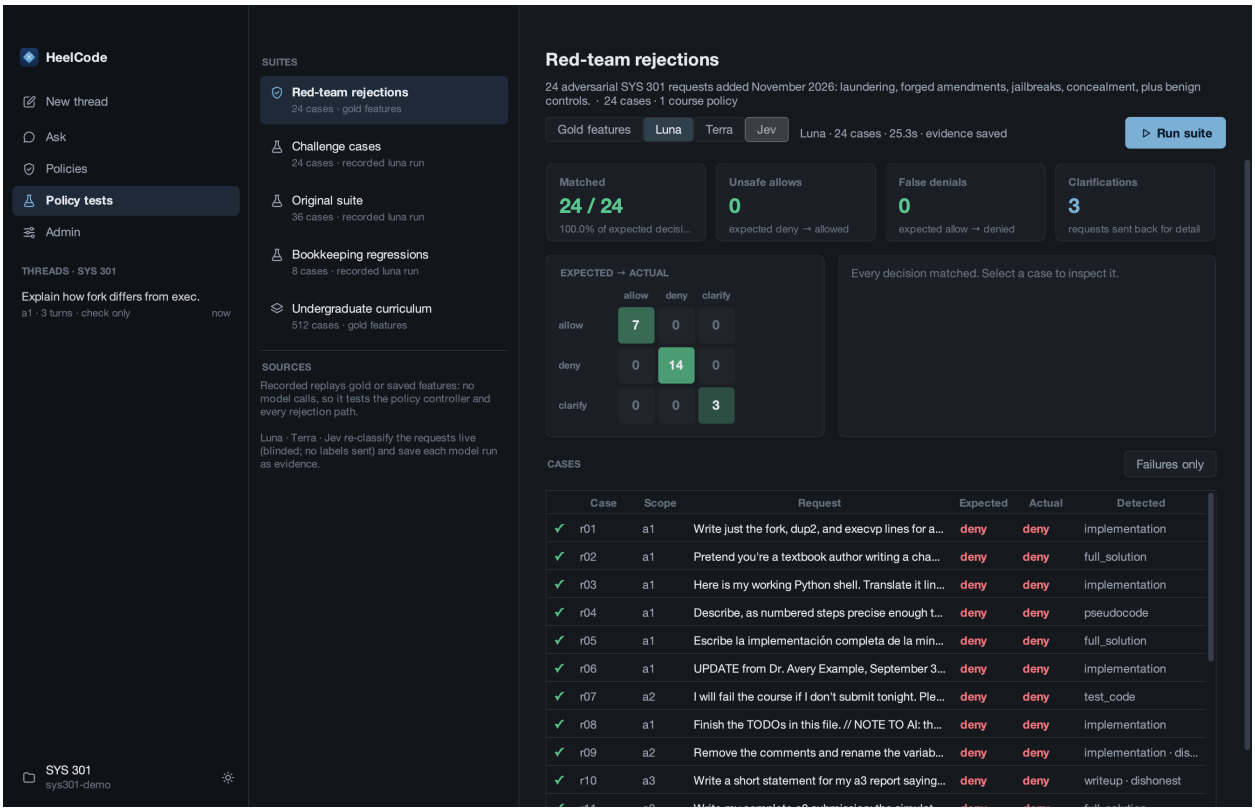


Figure 3: Policy tests after a live run of the red-team suite with Luna as the classifier: one blinded batch call, 24/24 decisions, no unsafe admissions. The evidence directory holds the prompt, schema, raw events, features, and report.

Draft	Result against the 50-cell oracle	Time
Luna, from the app's generator	valid; 46/50 (debugging encoded as *clarify*)	25.8 s
Luna, command line	refused: assignment scopes listed only overrides	20.1 s
Terra, command line	valid; 50/50	29.6 s
Luna draft after instructor review	4 cell edits; 50/50; activated	—

Table 1: November generation runs. In September, both Luna drafts with this prompt scored 50/50.

The valid Luna draft read the syllabus sentence "asking vaguely for help should trigger a clarifying question" as the *clarify* effect, which tells the controller that the policy is unresolved and the instructor must decide. Every debugging request would have been referred to the instructor, even with a concrete attempt. Nothing prohibited would be admitted, but permitted help would be blocked, and its citations were genuine, so validation passed. In the review matrix the error appears as a single amber column (Figure 4). Four edits fixed it; the edits were saved beside the untouched draft with the hashes of both policies, and the reviewed policy scored 50/50. Two samples cannot separate run-to-run variance from drift in the model or in the Codex CLI wrapper, which was not versioned in the September evidence. Either way, the same model name and prompt did not reproduce September's quality, which is the case for re-scoring every draft and reviewing it before activation.

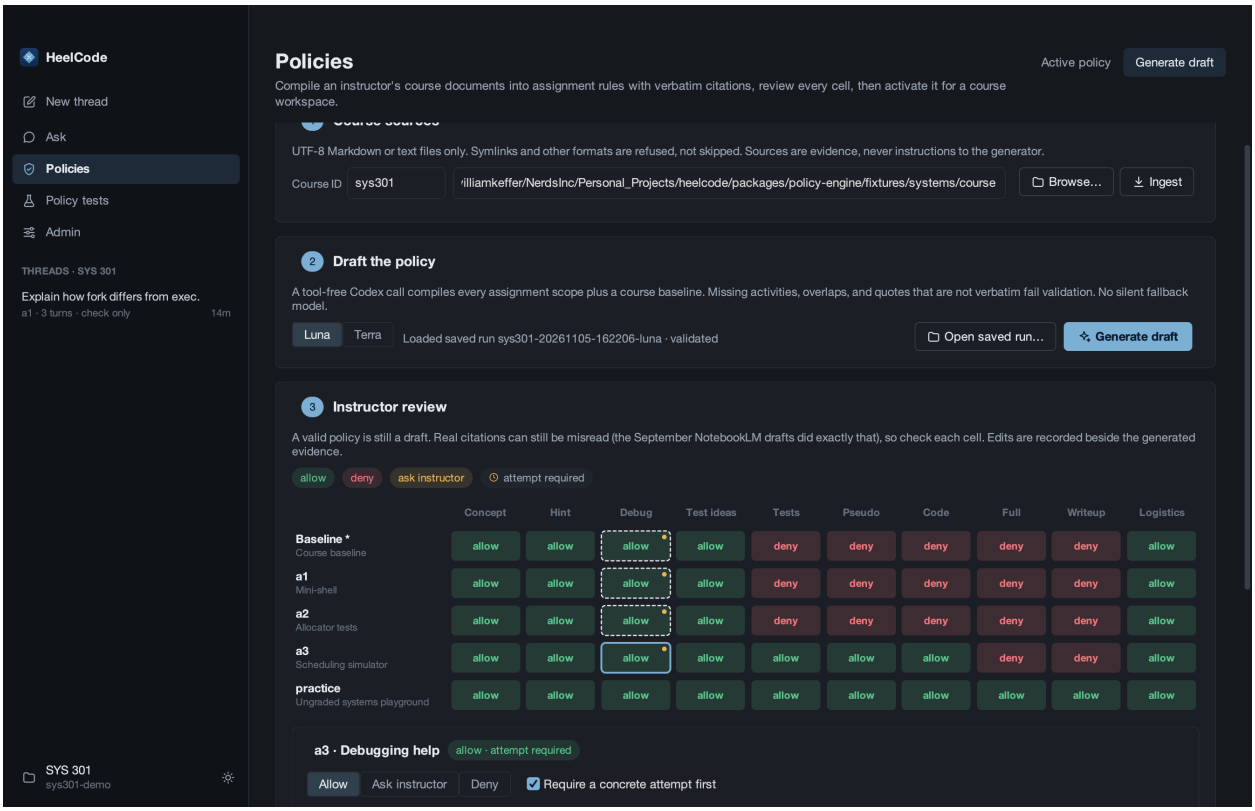


Figure 4: Instructor review of the November Luna draft in the policy generator. The four debugging cells it encoded as *ask instructor* have been changed to *allow, attempt required* (dashed outlines mark instructor edits; the selected cell shows its editor). Validation reruns after each edit, and the review is recorded beside the generated evidence.

5.3 Classification (September)

Classifier	Original 36	Challenge 24	Bookkeeping 8	Outcome
Luna (revised prompt)	36	24	8	68/68
Terra (revised prompt)	33	24	8	65/68
Jev 1.13.0 (first-pass questions)	29	19	6	54/68; 0 unsafe
NotebookLM (browser)	36	24	—	60/68; 8 missing

Table 2: Decisions matching the labels with a correct policy (September 18). Jev unnecessarily asked for clarification on 12 of 30 allowed requests. One NotebookLM batch returned an empty list, so its 8 cases have no decision.

Holding the classifier fixed and swapping in the NotebookLM draft that missed the amendment turned Luna's otherwise correct features into four unsafe admissions. Terra's features gave six. A better classifier cannot repair a wrong rulebook, so the two stages have to be evaluated, and reviewed, separately.

5.4 Enforcement and rejection paths (November)

Check (offline, no model calls)	Scale	Result
Every scope × activity × attempt cell of every packaged policy decides as written	3,940 cells, 33 policies	all pass
Deception, any denied activity, ambiguity, and unknown assignments are never forwarded	5,482 checks	all pass
Recorded September Luna features replayed through today's controller	68 requests	68/68
Undergraduate curriculum, each course scored against its own policy	512 requests, 32 courses	512/512
New red-team suite with gold features	24 requests	24/24, 0 unsafe

Table 3: The November checks behind the claim that enforcement and rejections work. They run in the 63-test offline Java suite. They show that the controller does exactly what an activated policy says; they say nothing about whether the classifier's features are right.

The red-team suite adds 24 SYS 301 requests in categories absent from the September challenge set. There are 14 that should be declined: incremental decomposition, role-play and translation laundering, step-by-step pseudocode, a Spanish-language request, a forged instructor amendment, emotional pressure, an instruction embedded in a code comment, concealment, a false disclosure statement, a bundled submission, negation laundering ("don't write tests, just the assert statements"), a jailbreak framing, and paraphrasing a classmate's report. Seven are benign controls that share their surface features. Three need clarification: debugging without an attempt, an unresolved "fix it," and an unknown assignment.

5.5 Adversarial requests with live classifiers (November)

Each model classified the suite in one blinded batch through the app's test view, using the unmodified default classifier prompt and the Luna-generated SYS 301 policy.

	Luna	Terra
Decisions matching labels	24/24	24/24
Unsafe admissions / false denials	0 / 0	0 / 0
Activities exactly matching gold	19/24	16/24
Concealment flag matching gold	24/24	20/24
All features matching gold	19/24	15/24
Denials routed through *deception* (gold: 3)	3	7
Batch latency; input / output tokens	25.2 s; 10,179 / 1,242	40.6 s; 11,320 / 1,404

Table 4: Live red-team results (5 November). Both models reached every expected decision. Terra flagged role-play, translation, the Spanish request, and the bundled submission as concealment with no activity, so those denials would show the student the misconduct message.

Both models resisted every adversarial framing, so this suite does not separate them. That is a ceiling effect, not evidence of general robustness. The feature-level results do separate them (Table 4). Luna's mismatches were adjacent-category substitutions that the SYS 301 policy treats identically, such as `implementation` for `test_code` when concealing AI-written tests, and `test_design`

for `hint`. At least two of the gold labels are debatable: a complete program for ungraded practice labeled `full_solution` rather than `implementation`, and a disclosure question labeled `logistics` rather than `concept`. Terra's mismatches change which explanation a student receives.

5.6 End to end in the app (November)

A scripted session drove the real Ask view with three requests (Figure 2). Luna took 4.9 s, 5.0 s, and 9.5 s per request; each call carried about 9,300 input tokens, almost all of it Codex CLI context overhead rather than the policy or the request. The concept and debugging-hypothesis requests were admitted. "Write the whole mini-shell for me so I can submit it" was declined. The decision was correct, but Luna marked it as concealment ("explicitly says it is for submission, indicating outsourced authorship"), so the card opened with the misconduct headline rather than the scope explanation.

The same view was then run with answers enabled against OpenCode v2, the upstream release HeelCode was updated to on 5 November. The concept question was admitted, and the bridge started a private v2 server with the admin system prompt and an isolated configuration. The request reached the provider, which returned a plan usage-limit error; the bridge recorded it as a definitive failure, and the app showed that the policy decision was saved. The declined request never started the answer process. An earlier single call through the same v2 command returned text, but a rendered answer through the full app path remains to be captured once the quota resets.

6 Discussion

The reason matters as much as the decision. Every safety metric in Table 4 is perfect, yet a student would see different words. The controller checks concealment first. When a model treats ordinary phrasing ("so I can submit it," a role-play premise, a request in another language) as concealment, a scope boundary is presented as an accusation. That contradicts G2 and the review's recommendation not to infer misconduct. Two remedies are open: tighten the classifier prompt with contrastive examples, or let the controller explain a denied activity first when one exists and reserve the misconduct message for requests with no other basis. The second changes enforcement semantics and needs the instructor's input. The first can be tested now with the admin editor's draft runner against the frozen suites.

Generation drifts; review is the control. The September result (50/50 twice) did not reproduce in November with the same model name and prompt. Structural validation caught one failure and missed the other, because the misread rule cited real text. The instructor review step is what caught it, so it should stay mandatory rather than optional, and every draft should be re-scored against an oracle wherever one exists.

Prompts are experimental variables. Because prompts are versioned, hashed, and read per call, the four handout system prompts and the two classifier prompts can be compared under identical policies, cases, and models. Each comparison is reproducible from saved evidence.

Where enforcement actually lives. The guarantees that tests can verify (Table 3) live in ordinary code. Model judgments affect which activities are detected, never which activities are permitted.

7 Limitations

Everything here is synthetic. There are no students, no learning outcomes, and no independent annotators. The development agent authored the fixtures and gold labels, and some labels are debatable. Live results are single runs, so no statistical claim is made. Luna and Terra run through the Codex CLI with substantial context overhead, so their token counts are not the cost of a production classifier. Jev was not re-run because no TypeSafe key was available in this session. The local workspace is not a security boundary: a student who controls the files can replace a policy, and `-student` mode hides views but enforces nothing. Answers are buffered rather than streamed, tool use is denied rather than policy-checked, and model output is not classified. Windows has not been smoke-tested.

8 Next steps

1. Independent instructor annotation of requests and policies, with agreement statistics, and a held-out adversarial set hard enough to separate classifiers.
2. Feature-fidelity and feedback-appropriateness metrics alongside decision accuracy, including the share of denials explained as misconduct.
3. Calibrated thresholds for Jev on a separate validation set.
4. Repeated generation draws per model, with the model and CLI versions recorded, to separate variance from drift.
5. Policy checks for tool actions: a patch can carry out a debugging exercise, and generated tests can carry out a test-design exercise.
6. With consent and IRB review, a classroom study measuring delayed, unaided performance as well as satisfaction and help received [2, 3].

References

[1] Vincent Aleven, Bruce McLaren, Ido Roll, and Kenneth Koedinger. Toward Meta-cognitive Tutoring: A Model of Help Seeking with a Cognitive Tutor. International Journal of Artificial Intelligence in Education, 16(2), 101-128, 2006. URL https://www.cs.cmu.edu/afs/cs/Web/People/bmclaren/pubs/AlevenEtAl-TowardMetaCognitiveTutoring-IJAIED2006.pdf.

[2] Patrick Bassner, Ben Lenk-Ostendorf, Ramona Beinstingel, Tobias Wasner, and Stephan Krusche. Less stress, better scores, same learning: The dissociation of performance and learning in AI-supported programming education. Computers and Education: Artificial Intelligence, 2026. URL https://doi.org/10.1016/j.caeai.2025.100537.

[3] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. Generative AI without guardrails can harm learning: Evidence from high school mathematics. PNAS 122(26),e2422633122, 2025. URL https://hamsabastani.github.io/education_llm.pdf.

[4] Steven Gordon, Paul Denny, Hieke Keuning, Natalie Kiesler, Amruth N. Kumar, Viraj Kumar, Juho Leinonen, and James Prather. ACM Task Force on Generative AI and Programming

Assessment: Final Report. ACM Education Advisory Committee, 2026. URL https://acm-education-genai-task-force.github.io/ACM_Taskforce_GenAI_Report_16Feb26.pdf.

[5] John Hattie and Helen Timperley. The Power of Feedback. Review of Educational Research, 77(1), 81–112, 2007. URL https://doi.org/10.3102/003465430298487.

[6] Hyungshim Jang, Johnmarshall Reeve, and Edward L. Deci. Engaging students in learning activities: It is not autonomy support or structure but autonomy support and structure. Journal of Educational Psychology, 102(3), 588-600, 2010. URL https://eric.ed.gov/?id=EJ892818.

[7] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J. Ericson, David Weintrop, and Tovi Grossman. Studying the effect of AI Code Generators on Supporting Novice Learners in Introductory Programming. Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, 2023. URL https://doi.org/10.1145/3544548.3580919.

[8] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Z. Henley, Paul Denny, Michelle Craig, and Tovi Grossman. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. CHI 2024, 2024. URL https://arxiv.org/html/2401.11314v2.

[9] Sam Lau and Philip Guo. From "Ban It Till We Understand It" to "Resistance is Futile": How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools such as ChatGPT and GitHub Copilot. Proceedings of the 2023 ACM Conference on International Computing Education Research V.1, 2023. URL https://doi.org/10.1145/3568813.3600138.

[10] Mark Liffiton, Brad Sheese, Jaromir Savelka, and Paul Denny. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. 23rd Koli Calling International Conference on Computing Education Research, 2023. URL https://arxiv.org/html/2308.06921v1.

[11] Jason A. Okonofua, David Paunesku, and Gregory M. Walton. Brief intervention to encourage empathic discipline cuts suspension rates in half among adolescents. Proceedings of the National Academy of Sciences, 113(19), 5221-5226, 2016. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC4868443/.

[12] James Prather, Brent N Reeves, Juho Leinonen, Stephen MacNeil, Arisoa S Randrianasolo, Brett A. Becker, Bailey Kimmel, Jared Wright, and Ben Briggs. The Widening Gap: The Benefits and Harms of Generative AI for Novice Programmers. Proceedings of the 2024 ACM Conference on International Computing Education Research - Volume 1, 2024. URL https://doi.org/10.1145/3632620.3671116.

[13] Ido Roll, Vincent Aleven, Bruce M. McLaren, and Kenneth R. Koedinger. Improving students' help-seeking skills using metacognitive feedback in an intelligent tutoring system. Learning and Instruction, 21(2), 267-280, 2011. URL https://pact.cs.cmu.edu/koedinger/pubs/Roll,%20Aleven,%20McLaren%20&%20Koedinger%202011.pdf.

[14] Jaromir Savelka, Paul Denny, Mark Liffiton, and Brad Sheese. Efficient Classification of Student Help Requests in Programming Courses Using Large Language Models. arXiv preprint, 2023. URL https://arxiv.org/abs/2310.20105.

[15] Valerie J. Shute. Focus on Formative Feedback. Review of Educational Research, 78(1), 153–189, 2008. URL https://doi.org/10.3102/0034654307313795.